



STEAM REVIEW ANALYSIS: SENTIMENT CLASSIFICATION, ASPECT-BASED ANALYSIS AND GAME RECOMMENDATION

Team members: HE181627 - Le Sy Minh Quang
HE176334 - Le Thanh Vinh
HE181673 - Tran Vu Minh

Supervisor: Mr. Bui Van Hieu

Bachelor of Artificial Intelligence
FPT University - November 2026

Acknowledgements

First and foremost, we would like to express our sincere gratitude to our instructor, Mr. Bui Van Hieu, for the guidance and feedback provided throughout this project. The suggestions received during the progress presentations, in particular the advice to group games by genre and publisher for recommendation and to explore keyword-based and transformer-based models, shaped a large part of the work reported here.

We are also thankful to the lecturers of FPT University, Hoa Lac campus, whose courses provided the foundations in machine learning and deep learning that this project builds on.

Finally, we thank our families and friends for their continuous support and encouragement during our studies.

Abstract

User reviews on Steam combine free text with a single binary verdict (*Recommended* or *Not Recommended*). The verdict does not say which parts of a game players liked or disliked, and a noticeable share of reviews are jokes, sarcastic, or contradict their own verdict. This project builds a set of models on 10,000 English Steam reviews of 99 games to (1) classify review sentiment from text, (2) flag suspicious reviews whose content contradicts their verdict, (3) predict whether a review will receive helpful votes, (4) detect which aspects of a game a sentence talks about (graphics, audio, gameplay, story, performance, price, online, developer) and whether each aspect is praised or criticised, and (5) recommend games from the same genre or publisher group.

The baseline models are TensorFlow/Keras networks (embedding with pooling, BiLSTM, 1D CNN, TF-IDF with dense layers and an ensemble), compared against logistic regression baselines under two evaluation scenarios: known games and unseen games. The best Keras model reaches a ROC-AUC of 0.902/0.905, on par with TF-IDF logistic regression (0.903/0.906). Fine-tuning DistilBERT raises the ROC-AUC to 0.937/0.930, and the 95% confidence interval of the gain excludes zero in both scenarios. For aspect-based analysis, about 4,600 sentences were labelled following written guidelines. Feeding keyword hints into DistilBERT and scoring each aspect with a pairwise model improves end-to-end F1 on a held-out test set from 0.451 to 0.622, and aspect polarity accuracy from 0.742 to 0.898. Sentences that praise one aspect and criticise another remain unsolved (accuracy about 0.55). The models are packaged into a local web application for review moderation and game recommendation. The aspect labels were drafted by a large language model following written guidelines and double-checked by the team.

Keywords: Sentiment Analysis; Aspect-Based Sentiment Analysis; Steam Reviews; TensorFlow; Keras; DistilBERT; Recommendation

Contents

1 Introduction **6**
- 1.1 Background and Motivation . . . 6
- 1.2 Literature Review . . . 7
- 1.3 Problem Statement . . . 7
- 1.4 Objectives and Contributions . . . 8
- 1.5 Scope and Limitations . . . 8
- 1.6 Structure of the Report . . . 8

2 Theoretical Background **9**
- 2.1 Sentiment Classification and Text Representation . . . 9
- 2.2 Neural Networks for Text . . . 10
- 2.3 Transformers and DistilBERT . . . 10
- 2.4 Aspect-Based Sentiment Analysis . . . 10
- 2.5 Evaluation Metrics . . . 11

3 Data Processing **12**
- 3.1 Dataset Description . . . 12
- 3.2 Data Preprocessing and Cleaning . . . 13
- 3.3 Exploratory Data Analysis and Feature Extraction . . . 14
- 3.4 Manual Labelling . . . 15
- 3.5 Data Splitting Strategy . . . 15

4 Methodology **17**
- 4.1 Overall Workflow . . . 17
- 4.2 Review-Level Models with TensorFlow/Keras . . . 18
- 4.3 Transformer Models . . . 19
- 4.4 Game Recommendation . . . 20
- 4.5 Hyperparameters and Implementation . . . 21

5 Results Analysis **22**
- 5.1 Experimental Setup . . . 22
- 5.2 Task 1: Review Sentiment Classification . . . 22
- 5.3 Task 2: Suspicious Review Detection . . . 24
- 5.4 Task 3: Helpfulness Prediction . . . 24
- 5.5 Task 4: Aspect-Based Sentiment Analysis . . . 25
- 5.6 Task 5: Game Recommendation and Demo Application . . . 27
- 5.7 Discussion . . . 28

6 Conclusion **29**
- 6.1 Summary of Achievements . . . 29
- 6.2 Limitations . . . 30
- 6.3 Future Work . . . 30

List of Figures

3.1 Share of positive reviews by playtime and by review length. 15

4.1 Overall workflow of the project. 17
4.2 Two-branch architecture shared by the Keras models. 18
4.3 End-to-end aspect-based sentiment pipeline. 20

5.1 Distribution of suspicion scores (log scale) and games with the highest share of mismatched reviews. 24
5.2 Share of negative sentences (%) per aspect for the 15 games with the most sentences; "–" marks fewer than 5 sentences. 27
5.3 Recommendation page of the demo application (Vietnamese interface) for a user who liked Stardew Valley, with the genre/publisher graph. 28

List of Tables

3.1 Fields of the Steam reviews dataset . . . 12
3.2 Cleaning steps for the Steam reviews dataset . . . 14
3.3 Manually labelled data . . . 16
3.4 Data splits for review-level tasks . . . 16

4.1 Training configuration . . . 21

5.1 Sentiment classification on the test sets of scenario A (known games) and B (unseen games) . . . 23
5.2 ROC-AUC on the 500 manually labelled test reviews . . . 23
5.3 Suspicious reviews among the 9,971 cleaned reviews . . . 24
5.4 Predicting whether a review receives at least one helpful vote (test set A) 25
5.5 Aspect detection (micro F1, mean over seeds). The variant selected on dev is in bold. . . . 25
5.6 Aspect polarity accuracy compared with whole-sentence models . . . 26
5.7 End-to-end aspect sentiment pipeline (a pair is correct only if both aspect and polarity are correct) . . . 27

6.1 Summary of the main results . . . 29

Chapter 1

Introduction

Steam is the largest digital distribution platform for PC games, and its user reviews are a main source of information for players deciding what to buy and for developers trying to understand how their game is received. Each review consists of a free-text body and a binary verdict: the reviewer either *recommends* the game or does not. Steam aggregates these verdicts into summary labels such as "Mostly Positive".

The verdict is a coarse signal. It tells whether a player recommends a game, but not *why*: one review may praise the graphics, criticise the crashes and still recommend the game. Moreover, many reviews are not serious evaluations at all: jokes, memes, sarcasm ("10/10 would never play again") or texts that contradict the button the user pressed. Such reviews distort both the aggregated score and any analysis built on it.

This project studies what can be learned from the text of Steam reviews beyond the verdict. Using 10,000 English reviews of 99 popular games collected by the team through the Steam API, we build and evaluate models for review sentiment, suspicious review detection, helpfulness prediction, aspect-based sentiment analysis and game recommendation. As the course focuses on TensorFlow, the core models are implemented with TensorFlow/Keras; following the instructor's feedback, the project was then extended with keyword features and a pre-trained transformer (DistilBERT).

1.1 Background and Motivation

Sentiment classification of reviews is one of the most studied problems in natural language processing. Bag-of-words classifiers already perform well on movie reviews [1], recurrent [2] and convolutional [3] networks learn their own text representations, and pre-trained transformers such as BERT [4] have become the standard approach when labelled data are limited.

Game reviews are a particular domain. In a study of the reviews of 6,224 Steam games, Lin *et al.* [5] found that game reviews are relatively short, that the playtime before a

review is a useful attribute specific to games, and that negative reviews carry most of the concrete complaints. Players use slang and inside jokes, mix praise and criticism, and often comment on patches and developers rather than on the game itself. This motivates two directions beyond plain sentiment classification: separating suspicious reviews from genuine ones, and identifying *which aspects* of a game a review discusses. These tasks match practical needs: a platform can down-weight reviews that contradict their verdict or promote likely helpful reviews early, and a developer can see which aspects attract most criticism without reading hundreds of reviews. The instructor also suggested grouping games by genre and publisher to recommend similar games to players.

1.2 Literature Review

Review-level sentiment. Pang *et al.* [1] framed review sentiment as text classification with bag-of-words features and linear classifiers. Neural approaches replaced hand-crafted features with learned embeddings processed by LSTMs [2] or 1D convolutions [3]. The transformer [6] and large-scale pre-training [4] made it possible to fine-tune a general language model on a few thousand examples; DistilBERT [7] keeps most of BERT's accuracy with 40% fewer parameters, which makes it practical on consumer GPUs.

Aspect-based sentiment analysis (ABSA). The SemEval-2014 shared task [8] standardised ABSA into aspect category detection and aspect polarity on restaurant and laptop reviews. Sun *et al.* [9] showed that BERT handles ABSA well when the aspect is given to the model as an auxiliary sentence; our pairwise polarity model follows this idea.

Sarcasm. Riloff *et al.* [10] describe sarcasm as a positive sentiment expressed about a negative situation, exactly the case where word-based classifiers fail and the text contradicts the reviewer's verdict.

Research gap. Public ABSA benchmarks cover restaurants or laptops; game reviews have other aspects (performance, monetisation, online community, developer support) and more humour and sarcasm.

1.3 Problem Statement

Given an English Steam review (with playtime and verdict where available), the project addresses five problems:

1. **Sentiment classification:** predict whether the review is positive or negative from its content.

2. **Suspicious review detection:** without labelled spam data, flag reviews whose content contradicts their verdict (sarcasm, trolling, misclicks) or that fail quality

rules.

3. **Helpfulness prediction:** estimate whether a review will receive a helpful vote, controlling for how long it has been online.

4. **Aspect-based sentiment analysis:** detect which of eight aspects a sentence mentions and whether each is praised or criticised, then aggregate per game.

5. **Game recommendation:** group games by genre and publisher and recommend games from the same group to users who liked a game.

1.4 Objectives and Contributions

The objective is a reproducible pipeline that turns raw reviews into useful signals. Beyond solving the five tasks, the project documents the sampling biases of the data, compares five TensorFlow/Keras models with logistic regression baselines on known and unseen games, measures the gain of DistilBERT with confidence intervals and manual error analysis, evaluates the aspect pipeline on a test set labelled after the keywords were frozen, and packages the models into a local web application. The contribution is a careful comparison rather than a new architecture: which model helps, by how much, on which reviews, and where errors remain.

1.5 Scope and Limitations

The project covers English reviews of 99 games, binary sentiment and eight predefined aspects. The recommendation uses a small illustrative dataset with one review per user, so it is demonstrated but not quantitatively evaluated. Aspect labels were drafted by a large language model following written guidelines and double-checked by the team. Transformers run on Keras 3 with the PyTorch backend because recent TensorFlow releases do not support GPUs on native Windows. Chapter 6 discusses these limitations.

1.6 Structure of the Report

Chapter 2 introduces the concepts and models used. Chapter 3 describes the data, cleaning, exploration and labelling. Chapter 4 presents the workflow, architectures and training setup. Chapter 5 reports and analyses the results, and Chapter 6 concludes.

Chapter 2

Theoretical Background

This chapter introduces the concepts used in the project: text representation, the neural architectures for sentiment classification, transformers, aspect-based sentiment analysis and the evaluation metrics.

2.1 Sentiment Classification and Text Representation

Review sentiment is formulated as binary classification. Each review x has a label $y \in \{0, 1\}$, where $y = 1$ means the reviewer recommended the game. A model outputs $\hat{p} = P(y = 1 \mid x)$ and is trained to minimise the binary cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]. \tag{2.1}$$

The same loss is used for helpfulness ($y = 1$ if the review has a helpful vote) and, with one output per aspect, for aspect detection.

Classical models represent a text as a sparse vector of term weights. TF-IDF weights a term t in document d by its frequency and its rarity in the collection, $\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log(N/\text{df}(t))$, where N is the number of documents and $\text{df}(t)$ the number of documents containing t. Word n-grams ("not good") capture short-range negation and character n-grams tolerate spelling errors. Logistic regression then models $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with the sigmoid $\sigma(z) = 1/(1 + e^{-z})$; with L2 regularisation it is a strong baseline for review sentiment [1].

2.2 Neural Networks for Text

A neural text model maps each token to a dense vector through a learned embedding matrix. Three ways of turning the sequence of vectors into one review representation are used in this project:

- **Average pooling** takes the mean of the token vectors; it is fast but ignores word order.
- **LSTM** [2] reads the tokens in order and uses input, forget and output gates to decide what to keep in a memory cell, so it can capture negation and long-range context. A bidirectional LSTM runs one LSTM in each direction and concatenates their final states.
- **1D convolution** [3] slides filters over the sequence to detect local phrases such as "not worth" wherever they occur, and keeps the strongest response of each filter by global max pooling.

To limit overfitting, dropout randomly zeroes activations during training and early stopping keeps the weights with the best validation loss.

2.3 Transformers and DistilBERT

The transformer [6] replaces recurrence by self-attention. With queries Q, keys K and values V,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{2.2}$$

so every token attends directly to every other token. BERT [4] pre-trains a deep transformer encoder on large text corpora by predicting masked words, and is then fine-tuned on a downstream task by adding a small output layer. Its input format `[CLS] text A [SEP] text B [SEP]` makes it natural to classify pairs of texts. DistilBERT [7] is a 6-layer version of BERT obtained by knowledge distillation; it is 40% smaller and 60% faster while keeping 97% of BERT's language understanding, which makes it the largest model the team could fine-tune on a 4 GB GPU. Long inputs must be truncated, and keeping both the beginning and the end of a text works better than keeping only the beginning [11].

2.4 Aspect-Based Sentiment Analysis

Aspect-based sentiment analysis (ABSA) decomposes an opinion into (aspect, polarity) pairs [8]. With a fixed set of aspect categories it has two steps: **aspect detection**,

a multi-label problem (a sentence may mention zero, one or several aspects), and **aspect polarity**, deciding for each detected aspect whether it is praised or criticised. A sentence-level model gives one polarity to the whole sentence and cannot handle "great graphics but the servers are terrible". Sun *et al.* [9] give the aspect to BERT as a second segment, `[CLS] aspect [SEP] sentence [SEP]`, so that the model scores that aspect only. We use this pairwise formulation for polarity, and a related idea for detection in which aspects proposed by keyword rules are given to the model as hints.

2.5 Evaluation Metrics

ROC-AUC is the main metric for binary tasks. It is the probability that a random positive example receives a higher score than a random negative one (0.5 is random guessing) and, unlike accuracy, it does not depend on a decision threshold. For aspect detection, precision $P = TP/(TP+FP)$, recall $R = TP/(TP+FN)$ and $F_1 = 2PR/(P+R)$ are computed by summing true positives, false positives and false negatives over all aspects (**micro F1**). In the end-to-end evaluation, a prediction is correct only if both the aspect and its polarity match the gold label.

Confidence intervals. Test sets contain from a few hundred to about two thousand examples, so differences between models must be checked against sampling noise. We use the paired bootstrap: the test examples are resampled with replacement 2,000 times, the difference between two models is computed on each resample, and the 2.5th and 97.5th percentiles form a 95% confidence interval (CI). If the CI excludes zero, the difference is unlikely to be due to the particular test sample.

Chapter 3

Data Processing

This chapter describes the two datasets, how they were collected and cleaned, what the exploratory analysis revealed, and how additional manual labels were created. Several findings here, in particular the sampling biases, determine which features the models may use.

3.1 Dataset Description

3.1.1 Steam Reviews Dataset

The main dataset was collected by the team through the public Steam review API between 21 and 28 September 2026. For each of 99 popular games, the 50 most recent positive and 50 most recent negative reviews were downloaded, giving 10,000 reviews written between December 2025 and September 2026. Table 3.1 lists the fields; all reviews were identified as English by an automatic language detector.

Table 3.1: Fields of the Steam reviews dataset

Field	Description
`app_id`, `game_name`	Steam identifier and name of the game
`review_id`, `review_text`	Review identifier and raw review body
`voted_up`	Recommendation verdict, used as the sentiment label
`playtime_forever_hours`	Reviewer's total playtime at crawl time
`votes_helpful`	Number of "helpful" votes received
`timestamp_created`	Time the review was written

This collection procedure has two consequences that matter throughout the report:

- The positive/negative ratio is balanced **by design** (about 50/50 per game) and does not reflect each game's real reception, so per-game statistics can only be compared relatively.

- Negative reviews are rarer on Steam, so collecting 50 of them required going further back in time. The timestamp and the helpful votes (which accumulate over time) are therefore correlated with the label **because of the sampling**, not the content. Both are excluded from sentiment models; review age is used only as a control variable for helpfulness.

3.1.2 Data Stream v2 (Recommendation Data)

For the recommendation task suggested by the instructor, the data collection members of the team prepared a second file with 299 rows for 100 game entries, three reviewers per game from three tiers (users owning the most games, a mid-range user, users owning the fewest games). Each row contains games owned, reviews written, playtime, helpful and funny votes, the game's overall rating and a review excerpt in Vietnamese. The file shows signs of being simulated (regular text, exactly three reviewers per game, rounded numbers, eight games duplicated with identical rows), Steam IDs lost precision, and every user has a single review. The report therefore treats it as **illustrative data** for demonstrating recommendation, not as real reviews.

3.2 Data Preprocessing and Cleaning

3.2.1 Cleaning the Steam Reviews

The cleaning removes noise without discarding signals that carry sentiment (Table 3.2). Emoji and censored words are kept because they carry sentiment: the `<censored>` token appears mostly in negative reviews. A rule flagging long runs of one repeated character was tried and dropped because it mostly caught legitimate texts such as "——", "!!!!" or "yesss".

The final dataset has **9,971 reviews** (5,000 positive, 4,971 negative). The median review has 20 words and the mean 54.9 words, a strongly right-skewed distribution.

3.2.2 Cleaning Data Stream v2

Removing the duplicated games leaves **275 reviews of 92 games**. Reviewers received anonymous identifiers because the original Steam IDs were unusable, and the verdict was recovered from the wording of the rating field and checked against the text. Publishers were joined from an earlier team file (79 of 92 games) and completed manually; genres were assigned manually into 15 groups. The result has 69 publishers, 14 of them with at least two games. Survival (16 games) and builder/management (11) are the largest genres. In this illustrative data, 33% of the reviews from users owning the fewest games

Table 3.2: Cleaning steps for the Steam reviews dataset

Step	Method	Effect
De-duplication	Match texts after lower-casing and collapsing whitespace; keep the first copy; drop duplicate groups with conflicting labels	29 removed (25 duplicates, 4 conflicts)
Markup removal	Remove Steam BBCode tags (keep their content) and URLs; de-duplicate again	82 reviews modified
Normalisation	Replace the heart symbols of Steam's profanity filter by a `<censored>` token; keep emoji as separate tokens	No empty reviews
Quality flags	Truncated, repeated-word spam, too short, mixed language (flagged, not removed)	20 reviews flagged
Name fixes	Normalise misspelled game names	13 names

are negative, against 6.5% for the mid-range tier and none for the tier owning the most games.

3.3 Exploratory Data Analysis and Feature Extraction

The exploration looked for signals besides the text that relate to sentiment, and for artefacts of the data collection.

- **The text is the main signal.** The strongest non-text feature has a mutual information with the label of only 0.044 (maximum 0.693). The class vocabularies differ clearly: *fun, best, love* versus *censored, boring, worst, crashes*.
- **Longer reviews lean negative** (Spearman $\rho = -0.26$), with the same sign in 96 of 97 games.
- **Playtime has an inverted-U relation with sentiment** (Figure 3.1): reviewers with less than two hours are positive in only 14% of cases, against 57% for 10–50 hours. Very short playtime often means refunds or games that do not start.
- **Explicit "x/10" scores** are strong but appear in only 4.4% of reviews; question marks and censored words go with negative reviews.
- **Timestamp and helpful votes** correlate most with the label, but only because of the sampling.

From this analysis, ten **meta features** complement the text: the log word count, presence and value of an x/10 score, presence of a question mark, presence of a censored

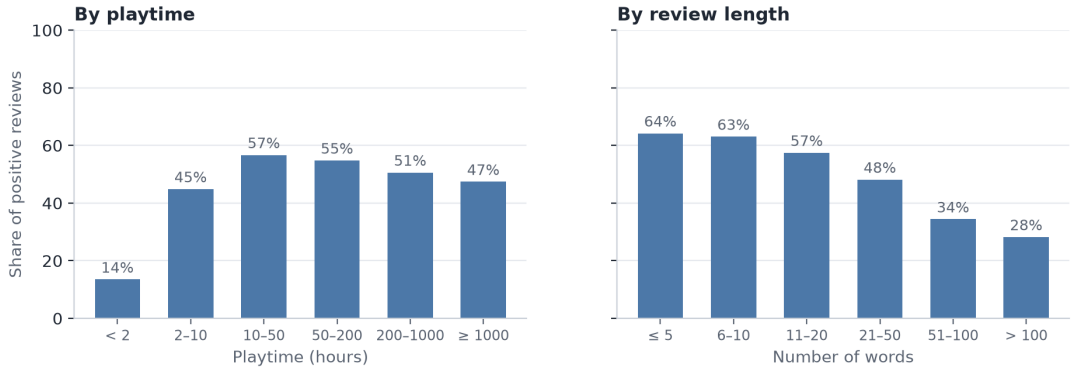


Figure 3.1: Share of positive reviews by playtime and by review length.

word, and playtime one-hot encoded into five bins (<2h, 2–10h, 10–50h, 50–200h, ≥200h). Playtime is binned because of its non-monotonic effect. Text features are learned by the models themselves.

3.4 Manual Labelling

The Steam verdict describes the whole review, so it cannot train or evaluate aspect-level models, nor tell whether an error comes from the model or from a misleading label. Additional labels were therefore created by a large language model (Claude) following written guidelines, without access to the Steam label, the game name or model predictions. The labels were then **double-checked by the team** against the guidelines.

Review labels. 500 random reviews from test set A were labelled with a sentiment (217 negative, 187 positive, 71 unclear, 25 mixed) and a type (393 normal, 76 jokes or memes, 15 too short, 8 sarcastic, 8 off-topic). For reviews with a clear sentiment, the label agrees with the Steam verdict in 94.8% of cases.

Sentence labels. Sentences were labelled with (aspect, polarity) pairs over eight aspects: graphics, audio, gameplay, story, performance/bugs, price/monetisation, online/community and developer; polarity is positive, negative, neutral or unclear. Generic sentences such as "great game" get no aspect. Table 3.3 lists the sets. The keyword lists used as a baseline were written after reading Test 1, which favours them on that set, so Test 2 was labelled only after the keywords were frozen and is the main test set for aspect-level comparisons.

3.5 Data Splitting Strategy

Two scenarios are used for review-level tasks (Table 3.4). **Scenario A (known games)** is a stratified random 70/15/15 split. **Scenario B (unseen games)** holds out 20% of the *games* as the test set, which checks whether a model learns general ways of praising

Table 3.3: Manually labelled data

Set	Size	Source	Purpose
Reviews	500	Random, test set A	Error analysis
Sentences, Test 1	500	Random, test set A	Evaluation
Sentences, extra	110	Test set A, selected by keywords	More polarity examples
Sentences, Test 2	500	Random, test set A, labelled after keywords were frozen	Main evaluation
Sentences, dev	438	Random, outside the test set	Model selection
Sentences, train	2,913	Outside the test set: random, keyword-selected and conflicting-polarity	Training

and complaining rather than game-specific words such as character or map names.

Table 3.4: Data splits for review-level tasks

Scenario	Set	Reviews	% positive	Games
A (known games)	Train	6,979	50.2	99
	Validation	1,496	50.1	99
	Test	1,496	50.1	99
B (unseen games)	Train	6,813	50.0	79
	Validation	1,203	50.0	79
	Test	1,955	50.7	20

All splits use a fixed seed, so every model sees exactly the same data, and vocabularies, TF-IDF weights and scalers are fitted on the training set only. Sentence sets are split by review, and the code checks that no review contributes sentences to both training and dev or test data.

Chapter 4

Methodology

This chapter describes how the models are designed, trained and evaluated: the overall workflow, the Keras models for the review-level tasks, the transformer models for sentiment and aspect analysis, the recommendation method, the demo application and the training setup.

4.1 Overall Workflow

The project has three streams of work (Figure 4.1). The first builds review-level models with TensorFlow/Keras. The second, added after the instructor's feedback that the models were not deep enough, introduces manual labels, keyword lists and DistilBERT for sentiment and aspect analysis. The third builds the game recommendation on Data Stream v2.

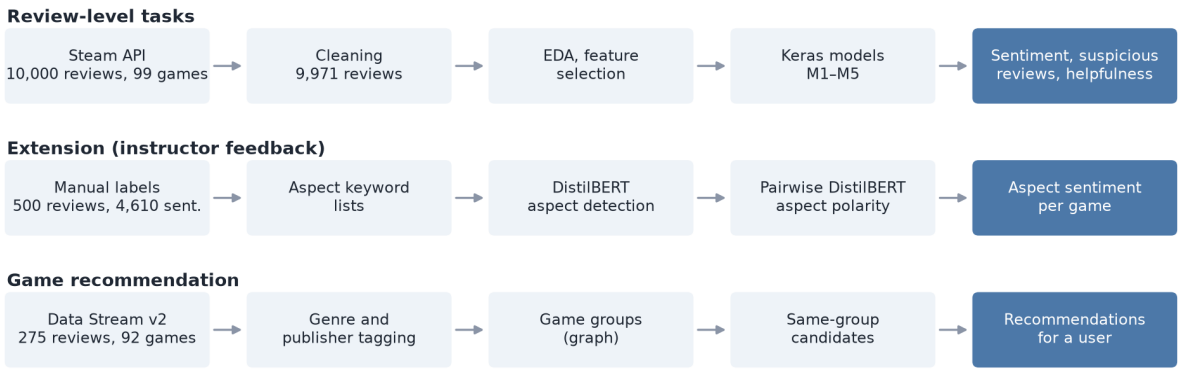


Figure 4.1: Overall workflow of the project.

In every experiment, hyperparameters, epochs and thresholds are chosen on the validation (dev) set and the test set is used only for reporting.

4.2 Review-Level Models with TensorFlow/Keras

4.2.1 Baselines

Three logistic regression (LR) baselines show how much information each input carries: **meta features only** (the ten features of Section 3.3); **TF-IDF + LR** with word 1–2-grams (up to 50,000 features) and character 3–5-grams (up to 80,000 features); and **TF-IDF + meta + LR**. The regularisation strength C is chosen from $\{0.25, \ldots, 8\}$ by validation ROC-AUC.

4.2.2 Keras Architectures

All Keras models share the two-branch structure of Figure 4.2, built with the functional API. In the **text branch**, a `TextVectorization` layer inside the model turns the raw string into token indices (vocabulary of 20,000 tokens, 256 tokens per review, which covers about 96% of reviews in full) or into a TF-IDF vector. The **meta branch** normalises the ten meta features. The branches are concatenated and followed by a dense layer, dropout and a sigmoid output. Because vectorisation is part of the model, a saved `.keras` file accepts raw text.

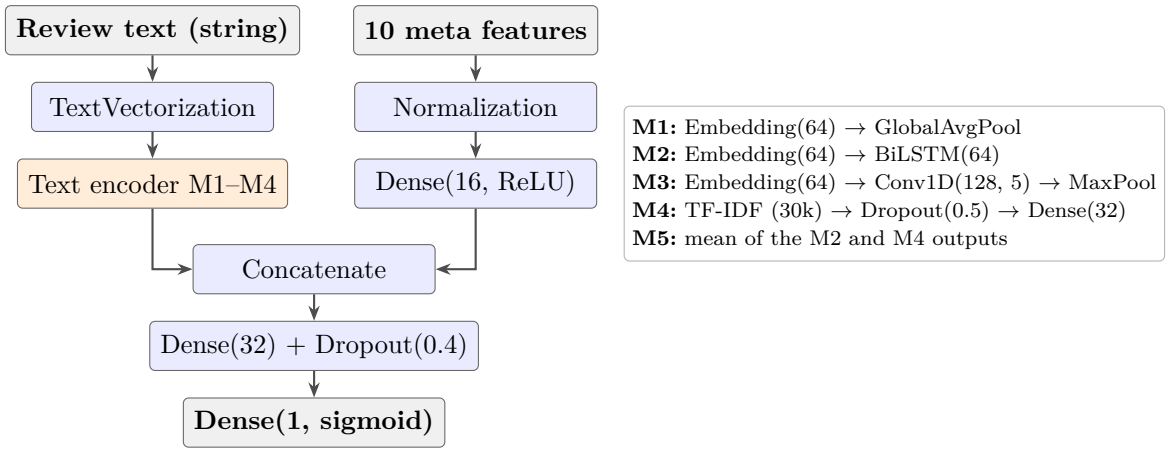


Figure 4.2: Two-branch architecture shared by the Keras models.

The models differ only in the text encoder (legend of Figure 4.2), following the text-classification labs of the course. M1 ignores word order, M2 captures order and negation, and M3 detects local phrases. M4 was added because M1–M3 did not beat the TF-IDF baseline: it brings n-gram TF-IDF features into a Keras network. M5 averages M2 and M4, which read the text differently (prediction correlation 0.85). Each model is trained separately for scenarios A and B.

4.2.3 Suspicious Review Detection

There are no labels for spam, trolling or sarcasm, so suspicious reviews are found from the disagreement between content and verdict, plus the quality rules (Algorithm 1). M4 is trained in 5-fold cross-validation so that every review is scored by a model that has not seen it.

Algorithm 1 Suspicious review detection

for each of 5 stratified folds k **do**
    Train M4 on the other folds; predict $p_i = P(\text{positive} \mid \text{text}_i)$ for reviews in fold k
end for
for each review i with Steam label y_i **do**
    $\text{mismatch}_i \leftarrow (y_i = 0 \wedge p_i \geq 0.9) \vee (y_i = 1 \wedge p_i \leq 0.1)$
    $\text{suspicion}_i \leftarrow |y_i - p_i|$ ▷ 0 = consistent, 1 = opposite
    $\text{suspicious}_i \leftarrow \text{mismatch}_i \vee$ any quality flag_i
end for
return suspicious reviews ranked by suspicion score

The thresholds 0.9 and 0.1 flag only reviews where the model is very confident; reviews with p close to 0.5 are more often genuinely mixed and are reported separately as "uncertain".

4.2.4 Helpfulness Prediction

The target is whether a review has at least one helpful vote (32.5% of reviews); a binary target is used because 67.5% of reviews have no vote. Older reviews have had more time to collect votes, so the log review age in days is added as a control variable, otherwise a model could learn that old reviews are helpful. Inputs are the text, the meta features, review age and the sentiment label. Four models are compared: LR on age only, LR on meta features and age, TF-IDF + meta + age with LR, and Keras M4 on the same inputs with class weights for the minority class.

4.3 Transformer Models

4.3.1 DistilBERT for Review Sentiment

The pre-trained `distilbert-base-uncased` [7] is fine-tuned through Keras 3 and KerasHub. Inputs are limited to 128 tokens; for longer reviews the first and last parts are kept [11], since reviewers often state their conclusion at the end. Token vectors are averaged (padding masked), followed by dropout 0.2 and a sigmoid output. The model uses the same splits as the Keras models, in both scenarios, with three seeds.

4.3.2 Aspect Detection

Aspect detection has eight sigmoid outputs, one per aspect, each with its own threshold chosen on dev. Five methods are compared: **keywords** (hand-written regular expressions per aspect, e.g. `crash|lag|fps` for performance), **TF-IDF + LR** (one balanced logistic regression per aspect), **DistilBERT** fine-tuned with eight outputs, the **union** of keywords and DistilBERT, and **keywords-in-input**. In the last method, the aspects found by the keywords are written before the sentence, `[CLS] graphics price [SEP] sentence [SEP]`, and DistilBERT decides whether to trust these hints, combining the precision of keywords with the transformer's ability to recognise paraphrases. To measure the effect of labelled data, keywords-in-input is trained on 1,265, 1,995, 2,725 and 2,913 sentences, and plain DistilBERT on the smallest and largest sizes.

4.3.3 Pairwise Aspect Polarity

For each (sentence, aspect) pair the model receives `[CLS] aspect [SEP] sentence [SEP]`, with the aspect as one word ("graphics", "performance", …), and predicts whether that aspect is praised or criticised [9]. A sentence with two aspects thus gets two independent predictions. The encoder is initialised from a DistilBERT sentiment model fine-tuned on Steam labels, so it starts with knowledge of game-review sentiment. To target the hardest case, one training variant adds 188 sentences with a contrastive connective ("but", "however") and at least two aspects. The baselines give every aspect of a sentence the sentence's polarity, predicted by M4 or by sentence-level DistilBERT.

4.3.4 End-to-End Aspect Pipeline

A review is split into sentences at sentence-ending punctuation and line breaks (keeping sentences of at least three words); each sentence goes through aspect detection, every detected aspect is scored by the pairwise model, and the pairs are aggregated per game (Figure 4.3).

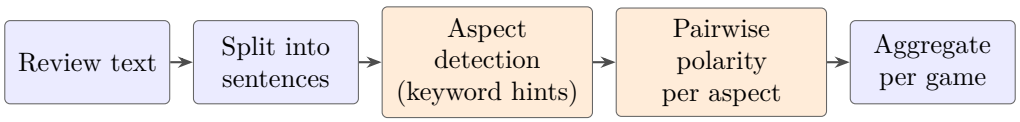


Figure 4.3: End-to-end aspect-based sentiment pipeline.

4.4 Game Recommendation

Every user in Data Stream v2 wrote one review, so collaborative filtering, which needs users who rated several games, is not possible. A content-based method is used instead:

games are compared by their attributes. Games, genres and publishers with at least two games form a graph in which each game is linked to its genre and publisher. For a user who liked game g, every other game h is scored as

$$\text{score}(h) = 2 \cdot \mathbf{1}[\text{genre}(h) = \text{genre}(g)] + \mathbf{1}[\text{publisher}(h) = \text{publisher}(g)], \tag{4.1}$$

with ties broken by the overall positive rating of h. Genre has the larger weight because players usually look for the same type of game. If the user disliked g, highly rated games from other genres are suggested.

4.5 Hyperparameters and Implementation

Table 4.1 lists the training settings. Keras models stop early when the validation loss has not improved for two epochs; transformers are scored on dev after each epoch and the best epoch is kept.

Table 4.1: Training configuration

Setting	**Keras models (M1–M4)**	**DistilBERT models**
Optimizer	Adam	AdamW, weight decay 0.01, gradient clipping 1.0
Learning rate	10^{-3} (3×10^{-4} for M4)	2×10^{-5} (3×10^{-5} for aspect detection), 10% warm-up, cosine decay
Batch size	64	16
Epochs	up to 15 (30 for M1), early stopping (patience 2)	3 (sentiment), 6 (aspect detection), 5 (polarity)
Input length	256 tokens, vocabulary 20,000	128 tokens, head + tail truncation
Regularisation	Dropout 0.4 (0.5 in M4), L2 10^{-4} in M4	Dropout 0.2
Seeds	1	3 (2 for some aspect variants)

The implementation uses Python 3.12 with pandas, NumPy and Matplotlib. All Keras models M1–M5 and the helpfulness model use **TensorFlow/Keras 2.20** on CPU (20–75 seconds per model), and **scikit-learn** provides TF-IDF, the LR baselines, splitting and metrics. DistilBERT is fine-tuned with **Keras 3 and KerasHub on the PyTorch backend** on an NVIDIA GTX 1650 (4 GB), because recent TensorFlow versions do not support GPUs on native Windows; the model code is standard Keras. A sentiment run takes about 10 minutes and the final aspect experiment about 2.5 hours.

The Keras models are served by a local **web application** (FastAPI and HTML/JavaScript) with pages for single-review analysis, batch moderation, a data overview and recommendation.

Chapter 5

Results Analysis

This chapter reports the results of every task in the order of the problem statement, then discusses them. All numbers are measured on test data that were not used for training or model selection.

5.1 Experimental Setup

Review-level models are evaluated on the test sets of scenario A (1,496 reviews of known games) and scenario B (1,955 reviews of 20 unseen games) described in Section 3.5, with ROC-AUC as the main metric and accuracy at a threshold of 0.5 as a secondary one. Aspect-level models are evaluated on the labelled sentence sets, with Test 2 as the main set. Transformer results are averaged over seeds, and a difference is considered reliable when its 95% paired-bootstrap CI does not contain zero.

5.2 Task 1: Review Sentiment Classification

Table 5.1 leads to four observations:

- **The text carries most of the information.** Meta features alone reach a ROC-AUC of about 0.71, TF-IDF + LR on the text 0.90; adding meta features to TF-IDF adds only 0.005.
- **Keras models trained from scratch do not beat TF-IDF + LR.** With fewer than 7,000 training reviews, M1–M3 start to overfit after 2–3 epochs. M4 is the best single Keras model, and the ensemble M5 matches the LR baseline (0.902 versus 0.903 in scenario A).
- **The models generalise to unseen games**: scores in scenario B are not lower than in scenario A, so the models learn general ways of praising and complaining rather than game-specific words.

Table 5.1: Sentiment classification on the test sets of scenario A (known games) and B (unseen games)

Model	Accuracy		ROC-AUC	
	A	B	A	B
Meta features only + LR	0.661	0.657	0.716	0.711
TF-IDF + LR	0.818	0.827	0.903	0.906
TF-IDF + meta + LR	0.825	0.836	0.908	0.909
M1 Embedding + Pooling	0.741	0.801	0.836	0.883
M2 BiLSTM	0.801	0.799	0.884	0.893
M3 CNN 1D	0.797	0.794	0.871	0.883
M4 TF-IDF + Dense	0.812	0.815	0.891	0.893
M5 Ensemble (M2 + M4)	0.824	0.826	0.902	0.905
DistilBERT (mean of 3 seeds)	≈0.86	≈0.855	**0.937**	**0.930**

- **DistilBERT is the best model**, with a ROC-AUC of 0.937 (A) and 0.930 (B). The 95% CI of its gain over TF-IDF + LR is $[+0.026, +0.047]$ in scenario A and $[+0.017, +0.037]$ in scenario B, well above the variation between seeds (about 0.002).

Error analysis. M4 misses mild complaints more often than praise: on test set A, 25% of negative reviews are classified as positive but only 13% of positive reviews as negative, and accuracy drops from about 85% for reviews longer than 50 words to 73% for reviews of at most five words. The 500 manually labelled reviews show where the remaining difficulty lies (Table 5.2). Both models are very accurate on clear reviews; jokes, mixed and unclear reviews are much harder. Reading the errors, DistilBERT reduces genuine model errors from 46 to 26, while errors caused by a Steam verdict that contradicts the text stay almost constant (13 and 14). No model can fix those, and they are exactly what Task 2 tries to find. Sarcasm such as "Oh great, another update that breaks everything. Love it." is still read as positive by M4.

Table 5.2: ROC-AUC on the 500 manually labelled test reviews

Group of reviews	Reviews	M4	DistilBERT
All	500	0.887	0.931
Clear sentiment, normal reviews	365	0.942	0.968
Others (jokes, mixed, unclear, …)	135	0.672	0.758

5.3 Task 2: Suspicious Review Detection

The out-of-fold M4 predictions reach an accuracy of 0.82 and a ROC-AUC of 0.90, the same level as on the held-out test set, so they can serve as a measure of what the text says. Table 5.3 and Figure 5.1 summarise the result of Algorithm 1.

Table 5.3: Suspicious reviews among the 9,971 cleaned reviews

Criterion	Reviews	%
Quality rules (any flag)	20	0.20
Content–verdict mismatch	270	2.71
Negative text but recommended	170	1.70
Positive text but not recommended	100	1.00
Suspicious (rules or mismatch)	**290**	**2.91**
Uncertain model ($\lvert p - 0.5\rvert < 0.1$), reported separately	1,078	10.81

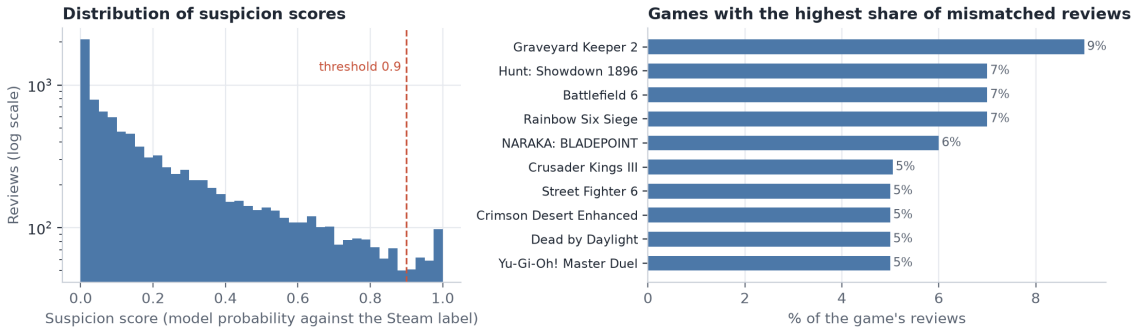


Figure 5.1: Distribution of suspicion scores (log scale) and games with the highest share of mismatched reviews.

The quality rules catch only 20 reviews, so the cleaned data contain little obvious spam; the mismatch criterion is more informative. Reading the flagged reviews reveals three groups: **sarcasm and trolling** ("10/10 would never play again", not recommended), **mixed reviews** that list strengths but conclude "I can't recommend it" or list bugs and still recommend, and **probable misclicks** with entirely positive text and a negative verdict. Several competitive online games are among those with the most mismatched reviews, but even for them the share stays below 10%. Without ground-truth labels the precision of the flags cannot be computed, so the output is meant as a ranked list for human moderators rather than an automatic filter.

5.4 Task 3: Helpfulness Prediction

Review age alone gives a ROC-AUC of 0.62, which confirms that it must be controlled for (Table 5.4). Meta features raise it to 0.77 and the text to 0.79, so content and length

Table 5.4: Predicting whether a review receives at least one helpful vote (test set A)

Model	F1	ROC-AUC	PR-AUC
LR, review age only	0.065	0.624	0.429
LR, meta features + age	0.515	0.771	0.601
TF-IDF + meta + age, LR	0.542	**0.786**	**0.636**
Keras M4 + meta + age (class weights)	**0.611**	0.761	0.605

help beyond age. Keras M4 has a slightly lower ROC-AUC but the best F1 (0.61) thanks to class weighting. The strongest factor is the sentiment label: 51.9% of negative reviews have a helpful vote against 13.2% of positive ones, followed by review length. Because negative reviews in this dataset are also older, part of this effect comes from the sampling.

5.5 Task 4: Aspect-Based Sentiment Analysis

5.5.1 Aspect Detection

Table 5.5: Aspect detection (micro F1, mean over seeds). The variant selected on dev is in bold.

Method	Train sentences	Dev	Test 2	Test 1
Keywords	0	0.546	0.557	0.640
TF-IDF + LR	2,913	0.605	0.602	0.602
DistilBERT	1,265	0.655	0.606	0.635
Keywords-in-input	1,265	0.692	0.640	0.696
Keywords-in-input	1,995	0.708	0.665	0.714
Keywords-in-input	**2,725**	**0.733**	**0.682**	**0.722**
Keywords-in-input	2,913	0.731	0.692	0.731
DistilBERT	2,913	0.729	0.679	0.715
Union of keywords and DistilBERT	2,913	0.722	0.687	0.723

Table 5.5 shows that:

- **The transformer clearly beats keywords and TF-IDF.** On Test 2 the selected model is 0.125 above the keywords (CI $[0.085, 0.169]$) and 0.080 above TF-IDF + LR (CI $[0.040, 0.120]$).
- **Keyword hints help most when labels are scarce:** +0.033 over plain DistilBERT with 1,265 sentences, but only +0.003 with about 2,900 sentences, where the CI contains zero ($[-0.017, 0.024]$).

- **More labels help with diminishing returns**: +0.025, then +0.017 per additional 730 sentences.
- **Keywords alone have low recall** (precision 0.735, recall 0.448 on Test 2). Their higher score on Test 1 (0.640) reflects that they were written after reading that set, which justifies Test 2 as the main test set.

Per aspect, gameplay gains most over keywords (F1 0.47 to 0.71), because players describe gameplay in many ways that are hard to list. Online/community and developer remain harder (0.64 and 0.68), and audio (3 test sentences) and graphics (21) have too few examples for reliable conclusions.

5.5.2 Aspect Polarity

On the 765 (sentence, aspect) pairs with a positive or negative gold label, the pairwise model reaches an accuracy of 0.898, against 0.689 for always predicting "negative" (Table 5.6).

Table 5.6: Aspect polarity accuracy compared with whole-sentence models

Whole-sentence baseline	Accuracy	Pairwise gain	95% CI
M4	0.742	+0.156	[+0.125, +0.188]
DistilBERT	0.835	+0.063	[+0.042, +0.085]

Two weaknesses remain. First, **sentences with conflicting aspects are not solved**: on the 55 pairs from sentences that praise one aspect and criticise another, all three models reach about 0.55 (ROC-AUC 0.58 for the pairwise model), and adding 188 such sentences to training changed nothing (0.552 versus 0.545). This was the case the pairwise formulation was meant to solve, so this objective was not met, although with 55 pairs the CI is wide ($[-0.145, +0.139]$ against M4). Second, **the model leans towards negative**: compared with M4, accuracy on negative pairs rises from 0.674 to 0.930, but on positive pairs it drops from 0.895 to 0.826.

5.5.3 End-to-End Pipeline and Results per Game

The end-to-end F1 rises from 0.451 to 0.622 (Table 5.7), almost entirely through recall (0.372 to 0.638) while precision stays similar: the final pipeline finds many more aspect opinions without proportionally more mistakes. Both test sets rank the four pipelines in the same order.

Applied to all 33,025 sentences of the 99 games, the pipeline produced 24,223 (aspect, polarity) pairs; since some sentences were used in training, this is a description rather than

Table 5.7: End-to-end aspect sentiment pipeline (a pair is correct only if both aspect and polarity are correct)

Pipeline	Test 2			Test 1
	P	R	F1	F1
Keywords + M4 (first version)	0.572	0.372	0.451	0.494
Keywords + whole-sentence DistilBERT	0.611	0.397	0.481	0.549
Keywords + pairwise DistilBERT	0.638	0.414	0.502	0.579
Keywords-in-input + pairwise DistilBERT	0.607	**0.638**	**0.622**	**0.640**

an evaluation. Gameplay is by far the most discussed aspect (10,297 mentions). Players are most negative about performance and bugs (90.3% of mentions) and developers (85.2%), and least about graphics (31.0%) and audio (43.7%). Figure 5.2 shows clear differences between games, the kind of summary a developer could use to prioritise fixes. The percentages should only be compared relatively, because of the 50/50 sampling per game, the longer negative reviews, the negative lean of the polarity model and the small number of sentences in some cells.

	Graphics	Audio	Gameplay	Story	Performance, bugs	Price, monetisation	Online, community	Developer
CONTROL Resonant	22	53	56	43	70	74	–	72
Wuthering Waves	18	50	62	44	83	81	90	81
Graveyard Keeper 2	0	15	69	40	100	70	100	73
WARDOGS	–	–	66	–	93	90	83	88
Crimson Desert Enhanced	32	–	53	64	93	54	57	70
No Man's Sky	42	–	66	62	97	69	61	74
RuneScape: Dragonwilds	24	–	66	38	86	62	48	92
Baldur's Gate 3	31	29	63	52	83	79	88	69
Counter-Strike 2	17	–	57	–	92	76	91	93
Monster Hunter Wilds	38	–	57	73	81	72	85	80
Total War: WARHAMMER III	50	–	69	36	96	83	69	86
Marvel Rivals	70	–	66	–	87	50	86	90
Aniimo	21	43	43	50	88	64	87	88
Diablo IV	11	–	59	50	88	87	86	82
Valheim	18	0	76	–	95	77	59	85

Figure 5.2: Share of negative sentences (%) per aspect for the 15 games with the most sentences; "–" marks fewer than 5 sentences.

5.6 Task 5: Game Recommendation and Demo Application

Figure 5.3 shows the recommendation page of the demo application for a user who liked *Stardew Valley*. Its genre is "Life & Job Sim" and its publisher has no other game in the data, so all candidates share the genre and are ranked by their positive rating (Euro Truck Simulator 2, Slime Rancher and Dave the Diver at 97%, then Slime Rancher 2 and

Supermarket Simulator). The graph groups games around genre and publisher nodes, as suggested by the instructor. Because each user has a single review and there is no record of what users played next, the recommendations cannot be evaluated quantitatively; the result demonstrates the method. The other pages of the application apply the Task 1–3 models: for example, a clearly positive review submitted as "Not recommended" receives a positive probability of 99% and is flagged as suspicious, and the batch page summarises all reviews of a game with the share of negative reviews by playtime and the most frequent words in negative reviews.

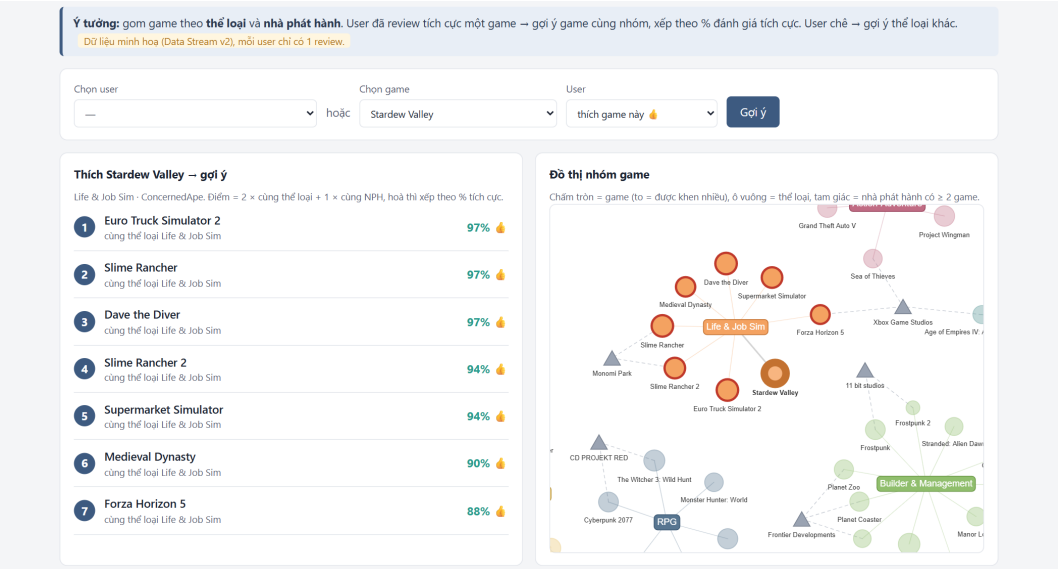


Figure 5.3: Recommendation page of the demo application (Vietnamese interface) for a user who liked Stardew Valley, with the genre/publisher graph.

5.7 Discussion

The results answer the instructor's question about deeper models with a nuanced picture. A pre-trained transformer gives a clear and statistically reliable improvement on every main metric (summarised in Table 6.1); models trained from scratch on 7,000 reviews do not. Keywords are most useful as hints inside the transformer and when few labels are available. Where the transformer did not help, namely sentences with conflicting aspects, a likely cause is that each aspect's polarity depends on the clause structure of the sentence, which the model does not learn from a few hundred examples. Finally, part of the remaining sentiment errors comes from the verdicts rather than from the models, and these reviews are precisely what the suspicious review detector is designed to surface.

Chapter 6

Conclusion

This project investigated what can be learned from the text of Steam reviews beyond the binary recommendation verdict.

6.1 Summary of Achievements

The data were cleaned into 9,971 reviews, and the exploration identified the text as the main signal and two sampling artefacts (timestamp and helpful votes) that would have leaked the label. Five Keras models trained from scratch reached a ROC-AUC of up to 0.902/0.905, on par with TF-IDF + LR and stable on unseen games. Out-of-fold predictions flagged 290 suspicious reviews (2.9%), mostly sarcasm, mixed reviews and probable misclicks, and text features raised helpfulness prediction from a ROC-AUC of 0.62 (age only) to 0.79. Following the instructor's feedback, DistilBERT and keyword hints were added; Table 6.1 summarises the resulting gains. Applied to all 99 games, the aspect pipeline shows that players criticise performance and developers most and graphics least. A genre/publisher recommender and a local web application complete the project.

Table 6.1: Summary of the main results

Task	**Baseline**	**Best model**	**Result**
Review sentiment, ROC-AUC (A / B)	TF-IDF + LR: 0.903 / 0.906	DistilBERT	0.937 / 0.930
Aspect detection, micro F1 (Test 2)	Keywords: 0.557	Keywords-in-input	0.682
Aspect polarity, accuracy	M4 whole sentence: 0.742	Pairwise DistilBERT	0.898
End-to-end pipeline, F1 (Test 2)	Keywords + M4: 0.451	Keywords-in-input + pairwise	0.622

The transformer improves every row of Table 6.1; for the first three rows the 95% CI of the gain over the baseline excludes zero (no CI was computed for the end-to-end

pipeline). Keywords help most as input hints and when labels are scarce. The pairwise model raises polarity accuracy overall but does not solve sentences with conflicting aspects. The project therefore shows not only that a deeper model helps, but by how much, on which reviews, and where it does not.

6.2 Limitations

- **Sampling.** The 50/50 sampling per game distorts sentiment ratios, helpful votes and timestamps; helpfulness results and per-game percentages should be read relatively.
- **Small groups.** Audio, graphics and conflicting-aspect sentences have few test examples; conflicting-aspect accuracy remains about 0.55 ($n = 55$).
- **Language and sarcasm.** Only English is handled, and mild sarcasm is often misclassified.
- **Recommendation data.** The recommendation uses illustrative data with one review per user and has no quantitative evaluation.
- **Environment.** Transformers run on Keras 3 with the PyTorch backend rather than TensorFlow, and are not yet integrated into the web application.

6.3 Future Work

1. **Targeted labelling** of audio, graphics and conflicting-aspect sentences; the learning curve suggests a few more points for aspect detection.
2. **Clause-level polarity:** split sentences at contrastive connectives before scoring, or use a model that attends explicitly to the aspect position.
3. **Deploy the transformer models** in the web application so developers can see aspect summaries for their own reviews.
4. **Better data:** reviews in natural proportions with real user identifiers, enabling realistic statistics, a fair helpfulness evaluation and collaborative filtering.

In summary, simple baselines, TensorFlow/Keras models and a fine-tuned transformer together turn Steam reviews into useful signals: what players think of a game, which reviews to distrust, and which aspects need attention.

References

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? sentiment classification using machine learning techniques," in *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2002, pp. 79–86. [Online]. Available: https://aclanthology.org/W02-1011/

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://ml.jku.at/publications/older/2604.pdf

[3] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751. [Online]. Available: https://aclanthology.org/D14-1181/

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT 2019*, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[5] D. Lin, C.-P. Bezemer, Y. Zou, and A. E. Hassan, "An empirical study of game reviews on the Steam platform," *Empirical Software Engineering*, vol. 24, no. 1, pp. 170–207, 2019. [Online]. Available: http://asgaard.ece.ualberta.ca/papers/Journal/EMSE_2018_Lin_An_Empirical_Study_of_Game_Reviews_on_the_Steam_Platform.pdf

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019. [Online]. Available: https://arxiv.org/abs/1910.01108

[8] M. Pontiki, D. Galanis, J. Pavlopoulos, H. Papageorgiou, I. Androutsopoulos, and S. Manandhar, "SemEval-2014 task 4: Aspect based sentiment analysis," in *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, 2014, pp. 27–35. [Online]. Available: https://aclanthology.org/S14-2004/

[9] C. Sun, L. Huang, and X. Qiu, "Utilizing BERT for aspect-based sentiment analysis via constructing auxiliary sentence," in *Proceedings of NAACL-HLT 2019*, 2019, pp. 380–385. [Online]. Available: https://aclanthology.org/N19-1035/

[10] E. Riloff, A. Qadir, P. Surve, L. De Silva, N. Gilbert, and R. Huang, "Sarcasm as contrast between a positive sentiment and negative situation," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013, pp. 704–714. [Online]. Available: https://aclanthology.org/D13-1066/

[11] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune BERT for text classification?" in *China National Conference on Chinese Computational Linguistics (CCL 2019)*, 2019, pp. 194–206. [Online]. Available: https://arxiv.org/abs/1905.05583